Spiff v2 — Research

What we read, what we took, and what it turned into. Working reference for the Spiff product team. Research carried out 30–31 August 2026; written 31 August 2026.

Executive summary

We studied one product deeply and five categories broadly, to answer one question: what does a non-technical person need to see before they trust a number an agent gives them?

The deep study is ThoughtSpot — the closest shipping analogue to Spiff, with a real governance model, real privilege names and unusually candid documentation. Six passes cover its search and conversational AI surface, Liveboards and sharing, the semantic layer, security and admin at scale, onboarding and brand, and the developer/embedding/MCP surface. Two further passes
cover ground ThoughtSpot does not: Claude Cowork and the agentic-chat vocabulary (ChatGPT canvas and agent, M365 Copilot, Gemini, Notion Agent, Perplexity Projects), and catalogues, business rules and access governance (Atlan, Collibra, Alation, Purview, Databricks, Select Star, Dataplex, dbt, Cube, Immuta, Camunda DMN, Great Expectations, Monte Carlo, Entra, SailPoint,
Okta, and the UK GSS on disclosure control).

Seven ideas changed the design, sharpest first.

1. The interpretation is the headline, and the diff is the explanation. ThoughtSpot titles every answer with the tokens it understood, not the sentence you typed, and highlights the token that changed on each follow-up. A wrong answer becomes visibly wrong before anyone reads the chart. v1 buried provenance in a footer; v2 puts the parsed question on top.
2. Show the plan before doing the work. ThoughtSpot's "Why" flow publishes an analysis plan before running anything; Cowork shows a numbered todo list you review and steer. Two products, two domains, one conclusion: the plan is both the explanation and the consent moment.
3. Sharing re-scopes, it never widens — and the dialog must say so. ThoughtSpot's default is the opposite; Cowork's live artifacts get it right — "Viewers use their own connector access, not the creator's" — the shipped precedent for Spiff's second invariant.
4. Trust needs four states, not two. Verified / Draft / Warning / Deprecated. Alation's split of warning from deprecated is the one most products get wrong. And the badge is only honest if editing breaks the seal.
5. Rules are sentences, with a named hit policy, tolerance, and a test harness. Immuta's builder reads "Mask columns tagged [TAG] using [METHOD] for everyone except [EXCEPTION]"; Camunda's DMN names what happens when two rules match; Great Expectations' mostly admits real
rules are ≥95%, not 100%. A rule nobody can test is a rule nobody will change.
6. Usage is the documentation. "287 people use this, mostly Regional Coordinators, and here are the three questions they ask" beats a data dictionary. Select Star's popularity, top users and popular joins cost nothing to render.
7. The tool list is the permission UI — and so is the read log. Show every tool in plain English before connecting; three states per tool; writes default to Needs approval. Alongside it, our two deliberate inversions: ThoughtSpot logs no view, query or download event and has no "view as user", so Spiff logs reads and makes preview-as-viewer a screen.

The result is eleven new v2 screens beside the v1 workspace, and a data spine (src/data/50-org.js) with 15 datasets, 32 named people, 12 groups, 8 role bundles and a four-state certification vocabulary that exists only because of this research.

A later pass, §12, answers a question the brief added afterwards — where the meaning of a dataset comes from before anyone writes it down — and produced two further screens, Sources and the understanding run. Its one-line finding: every product that reads meaning out of code and

documentation lets the machine write into a suggestion slot and makes a named person move it into
the fact slot.

Contents

1. [Why we studied ThoughtSpot — and what we did not copy](#1-why-we-studied-thoughtspot--and-what-we-did-not-copy)
2. [The search-and-answer experience](#2-the-search-and-answer-experience)
3. [The semantic layer: what a dataset must expose](#3-the-semantic-layer-what-a-dataset-must-expose)
4. [Trust and certification: why four states beat two](#4-trust-and-certification-why-four-states-beat-two)
5. [Governance at scale: groups, bundles, diffs](#5-governance-at-scale-groups-bundles-diffs)
6. [Business rules as sentences](#6-business-rules-as-sentences)
7. [Discovery for non-technical users](#7-discovery-for-non-technical-users)
8. [The portal-chat workspace](#8-the-portal-chat-workspace)
9. [MCP: the tool list is the permission UI](#9-mcp-the-tool-list-is-the-permission-ui)
10. [Onboarding: sample data first, your data last](#10-onboarding-sample-data-first-your-data-last)
11. [Where Spiff deliberately diverges](#11-where-spiff-deliberately-diverges)
12. [Code and documentation as the source of dataset meaning](#12-code-and-documentation-as-the-source-of-dataset-meaning)
13. [What we built as a result](#13-what-we-built-as-a-result)
14. [Confidence notes](#14-confidence-notes)
15. [Open questions](#15-open-questions)
16. [Appendix A — the canonical dataset profile page](#appendix-a--the-canonical-dataset-profile-page)
17. [Appendix B — the canonical business rule object](#appendix-b--the-canonical-business-rule-object)

---

1. Why we studied ThoughtSpot — and what we did not copy

ThoughtSpot makes the same three promises Spiff makes: ask in plain language, get a governed answer, share it without leaking anything. It brands itself the "ThoughtSpot Agentic Analytics Platform" — "Data to Decisions, Powered by Agents" — with one agent stem and four jobs: Spotter (AI Analyst), SpotterModel, SpotterViz, SpotterCode, plus AgentSpot.

Its most transferable claim is architectural, not statistical. Under "Deterministic Insights, Full Verifiability" it says Spotter "translates questions into search tokens grounded in your governed semantic layer—producing fully traceable, auditable queries." It answers the hallucination objection with a mechanism, not a percentage. Show the trace; do not assert the trust.

It is also a good teacher because it publishes its limits — Spotter cannot be coached on "agent tonality, narrative style, granular chart formatting" and "does not support parameters" — and a config flag, showSpotterLimitations, exists purely to "show limitation text of the spotter underneath the chat input".

What we deliberately did not copy:

- Tokenised search as the input surface. Its own phrasing guidance is anti-conversational — prefer "carroll alice" over "Find all books by Lewis Carroll", and "Type slowly, and use the suggestions." Tokens are a superb explanation layer and a poor keyboard for a locality secretary. We took the chips as output only.
- Permissive-by-default data access on share — see §11.

- Three coexisting agent generations with different data-sharing postures: Classic "does not share actual data values with large language models"; in Spotter 3 "data is always shared with LLMs". Honest, but a 428-person division should not have to reason about a version matrix.
- A split management estate — schedules at Data > Utilities, alerts at Insights → Monitor subscriptions. Two screens, one question.
- RBAC as a one-way door: off by default, requested through support, and "once enabled cannot be disabled."

Sources: <https://www.thoughtspot.com/>, /product/agents/spotter, <https://docs.thoughtspot.com/cloud/26.8.0.cl/spotter-limitations>, <https://developers.thoughtspot.com/docs/rbac>

---

2. The search-and-answer experience

Interpretation as the headline. Every Spotter answer is titled with search tokens representing the interpreted query, above the chart, with a table/chart toggle and an action row: Edit, Download (PNG / XLSX / CSV), Save, Reset. Tokens are colour-coded by grammatical role — measures green, attributes blue, filters grey — which teaches query structure without a tutorial: a user learns a thing is a filter because it is grey. They are also the control surface: selecting a chip offers alternatives, hovering reveals an x, you can click between chips to insert mid-query. "Show work" explains the interpretation and "More details" exposes column statistics.

The token diff is the highest-value, lowest-cost idea in the whole research effort. "Spotter treats successive questions in a conversation as a follow-up" — and highlight changes appear in the tokens for each new question. The user sees exactly which token changed between turns, which is the question people actually ask of a follow-up: did it keep my filter?

Plan before execute. Ask "Why did my sales drop last month?" and the answer arrives in three parts, in order: an analysis plan stating "which attributes it will analyze to find the answer" before anything runs; change-analysis charts that "break down the contributions from different attributes"; then a narrative summary of "the key drivers".

Depth as a dial. Search mode is "a high-speed calculator"; Deep analysis mode is "the mathematician who shows the entire proof". Auto mode drops the source picker and states the consequence in the answer — "sourced from Sales Data and Support Data" — with a confidence
score and a reason for the choice. Confidence with a reason, never a bare percentage.

Small things that carry weight. The submit icon becomes a stop button while generating (enableStopAnswerGenerationEmbed); disambiguation is sticky — "your choice is sticky… in the scope of the current search" — so the same clarification is never asked twice; and search-on-enter lets a user compose freely and commit deliberately.

Feedback that teaches, then a screen to manage it. "+ Add to Coaching" opens "Confirm reference question" → "Review business terms and mapped search tokens" → Done: feedback becomes a reviewable artefact, not a thumbs-up into a void, and needs its own permission.
The loop ships as the Spotter Conversations Liveboard — "Feedback Response" (upvote:downvote ratio), "Conversations by Model", "Downvoted conversations", a "Complete conversations log".

Chart stability over cleverness. A new query means ThoughtSpot "analyzes the data and automatically selects the most appropriate chart"; a follow-up "should retain the existing chart type unless constraints force it to change"; a drill "maintain[s] the current chart type wherever possible"; a chart needs "at least one attribute and one measure" or you get a table; and "Colors are maintained across searches within a session." Pick well once, then stop moving the furniture.

Sources: <https://docs.thoughtspot.com/cloud/26.8.0.cl/spotter-getting-started>, /spotter-why,

/spotter-versions, /spotter-auto-mode, /search-data, /search-bar, /charts, /spotter-conversations-liveboard, <https://developers.thoughtspot.com/docs/Interface_SpotterEmbedViewConfig>

---

3. The semantic layer: what a dataset must expose

ThoughtSpot's semantic object is a Model — "a logical view created on top of a more complex data model, to enable business users to more easily consume data" — edited through a tabbed editor whose IA is itself the lesson: Tables · Columns · Formulas · Filters · Parameters · Instructions · Settings. Columns live in a spreadsheet-like grid with ALL-CAPS headers, double-click to edit, Save Changes at top, multi-select for bulk edit. It scales to 200 columns with no modals.

The properties that matter to a non-technical audience are a short list.

Property	Why it matters
Column Type — ATTRIBUTE or MEASURE	Two axes only: the thing you group by, the number you do maths on
Additive (YES/NO)	Gates which aggregations are offered at all
Aggregation names	Shown in English — TOTAL OF, AVG OF, UNIQUE COUNT OF — never as SQL
Synonyms	The cheapest discovery feature in the product: "turnover, revenue, sales" in one box
Index Priority (1–10, default 1; 8–10 for important columns)	Search ranking as a designable property
AI Context (max 400 chars; auto-generation targets 150–250)	Prose telling the agent how to use the column
Format Pattern	Java notation: #,##0.## renders 12345.6789 as 12,345.68

Above the column sits Data model instructions — up to 10,000 characters of "global rules that guide how Spotter interprets and answers questions", strictly enforced, taking precedence over AI-generated memory. The published examples are plain sentences: "When I ask for
last month, use 'last 30 days' as a filter", "For counting customers, always use the unique count of 'Customer Cred ID'."

Three consequences we adopted.

- A single agreed metric definition needs no DSL. The most modern part of the product is a text box with an owner.
- Strict rules and learned memory are different trust levels and need different UI. Instructions are hand-written and enforced; memory is generated and contextual.
- Provenance should be permanent, not a hover. The "Last data refresh time" tooltip carries exactly the right payload — last refresh, creator, and sample values from that column — and is invisible until you hover. Sample values are the fastest trust signal a non-technical person has: GST-2024-0871 beats any type annotation.

Also worth taking: the Dependents tab lists "the names of the dependent objects (Models and Liveboards), and the columns they use" and blocks deletion with "Cannot delete because of dependent objects". And one thing to avoid — ThoughtSpot documents its own bug: Attribution Dimension is table-level, but "the current UI implies this is a column-level setting." Never render a property at a scope it does not belong to.

The field is converging on the same object. dbt separates semantic models from metrics, a metric carrying name, label, description, type (simple, ratio, cumulative, derived), filter, meta. Cube adds public: false ("cannot be queried through the API") and, tellingly, meta.ai_context. Looker's access_grant and access_filter restrict explores, fields and rows by user attribute — the definition is shared, the result set is per-viewer. That is Spiff's invariant, already proven in a shipping product.

Sources: <https://docs.thoughtspot.com/cloud/latest/semantic-layer>, /data-modeling-settings, /data-model-instructions, /spotter-ai-context, /search-data-refresh-time, <https://docs.getdbt.com/docs/build/metricflow-commands>, <https://docs.cube.dev/reference/data-modeling/measures>

---

4. Trust and certification: why four states beat two

ThoughtSpot ships a lifecycle, not a badge:

1. An admin grants the Can verify privilege; holders are verifiers (the blog calls them Data Stewards).
2. An author requests verification from the more-options menu, choosing which stewards to notify; they get an email with a direct link. The verifier clicks Approve or Decline.
3. "A blue verified label will be added to the top left of the Liveboard"; lists gain a sortable Verified column; Show Liveboard details reveals verifier name and date.
4. Editing breaks the seal: "a yellow warning banner… indicating it has been edited after verification", persisting until re-verified.

Step 4 is what makes the badge honest. A certification that survives edits is decoration.

Two structural choices are worth noting. They certify Liveboards, not Models — the thing people share, not the plumbing. We diverge: in Spiff the dataset is what a person is asked to trust before asking a question, so datasets carry the badge and answers inherit a line from the datasets they touch. And they publish recommendations rather than a score — Enable indexing, Fix date value issues, Fix column type mismatches. Actionable beats gradeable, though we kept a score too, because a non-technical reader wants one number before a punch list.

The four-state model comes from the catalogue vendors. Atlan carries "verified, draft, or deprecated". Alation ships three trust flags — Endorsement, Warning, Deprecation — set by Catalog Admins and Stewards, surfaced in search results as well as on the object page. Alation's split is the important one: warning means "this is risky right now", deprecated means "this is retired". Collapsing them loses the only state a user can act on today.

Spiff's CERT vocabulary has five entries — the standard four plus one the GST context needed.

State	Note shown in the UI
verified	"Checked and signed off by a named steward."
draft	"Usable, but definitions may still change."
warning	"Something is wrong right now. Read the note before using."
deprecated	"Retired. A replacement exists."
blocked	"Excluded from Spiff by policy."

blocked exists for one dataset — Pastoral care notes — listed in the catalogue with no fields, no rows, and the note "Excluded from Spiff by policy. Listed here so people stop looking for it." Showing that a thing is deliberately absent is more trustworthy than hiding it, and it is the clearest demonstration that "no data" and "not allowed to see the data" are different.

Sources: <https://docs.thoughtspot.com/cloud/latest/liveboard-verify>, <https://docs.atlan.com/product/capabilities/discovery/concepts/what-are-asset-profiles>, <https://docs.alation.com/en/latest/welcome/BestPractices/UseTrustFlagstoProceedwithConfidence.html>

---

5. Governance at scale: groups, bundles, diffs

ThoughtSpot's most scalable decision is a constraint. Roles bind to groups, not users — "Users inherit role privileges from the groups to which they are assigned" — RLS evaluates "for every row and group combination", and column security is blunt: "You cannot specify column

security rules at user level." Users in several groups get the union.

We copied it literally. Every entitlement binds to one of 12 groups, each carrying one or more of 8 role bundles; the person row is read-only for entitlements. Groups carry their own provenance — Synced · Directory, Attribute rule, Manual, Manual · reviewed monthly — because "why does this person have this?" is the question a reviewer actually asks.

Two ideas from the privilege list. Download is three separate privileges — DATADOWNLOADING, CAN_DOWNLOAD_VISUALS, CAN_DOWNLOAD_DETAILED_DATA — because a picture is not a dataset. And SHAREWITHALL gates seeing other users' names: it lets a user "see the names of and share with users outside of the groups the user belongs to." In a religious community's operations arm, the directory itself is information.

Bundles, not matrices. Entra's access package is "A bundle of all the resources with the access an identity needs to work on a project or perform their task", governed by a policy defining "who can approve, and how long they have access." Nobody at scale grants table by table. Spiff's equivalent is a named bundle — Locality Secretary, Regional Coordinator, Safeguarding Lead, National Office — with a plain-English description and a risk level. Entra supplies two further habits: grants are time-bound by default, and assignment is attribute-driven, revoking when the locality or role record changes.

The diff, not the list. SailPoint's review vocabulary is the most operational we found: Reassign, Approve, Delegate, Allow Exception (temporary access with an expiry), Revoke or Edit Access, Revoke Account — with a percentage complete bar and a mandatory Sign Off that cannot happen until every item is decided. At 428 users the reviewable unit is what changed since last time, with dormancy the highest-value column ("granted 14 months ago, never used"). One person in our fixture carries exactly that flag.

Requests should be a flow, not an email. ThoughtSpot turns three dead ends into requests — a Request Access page, a locked icon beside an object name, and a locked icon whose dropdown lists the required data sources — then notifies the owner, anyone with edit access, all cluster administrators and ThoughtSpot support, after which "any of the users who got the request then shares the Liveboard." No approval queue, no SLA, no expiry, no audit trail of grants.

Purview shows the finished version: Permitted access (usage purposes), Approval requirements, Attestations, sequential approval (manager → privacy reviewer → approvers → optional access provider), and four statuses — Pending / Declined / Approved / Completed — with the last two separated because provisioning can fail. Requesters track status under My data access; approvers work a Requests and approvals queue. Both screen names went straight into v2. Purview also supplies a warning to design away from: "The product doesn't enforce policies such as attestations… The data consumer attests that they'll follow these policies." Never let a UI claim access the user does not have.

Finally, the audit gap. ThoughtSpot logs auth, full CRUD across every object type, identity and entitlement changes, a single coarse SHARE_OBJECTS, ingest and tenancy — and logs the attempt beside the success (CREATE_CONNECTION_ATTEMPTED next to CREATE_CONNECTION), which is an excellent instinct. But there is no view, query or download event, retention is 30 days, and read activity lives in a separate set of monitoring Liveboards. That split-brain is the origin of Spiff's Activity log (§11).

Sources: <https://docs.thoughtspot.com/cloud/latest/groups-privileges>, /rbac, /security-rls, /share-request-access, /audit-logs, <https://learn.microsoft.com/en-us/entra/id-governance/entitlement-management-overview>, <https://learn.microsoft.com/en-us/purview/unified-catalog-data-product-access-policies>, <https://documentation.sailpoint.com/identityiq/help/certifications_and_access_reviews/access_review_decisions_operations.html>

---

6. Business rules as sentences

The best rule-authoring UI we found belongs to Immuta: a natural-language sentence with dropdowns.

> "Mask columns tagged [TAG] using [METHOD] for everyone except [EXCEPTION]."
> "Only show rows where [CONDITION] for everyone except [EXCEPTION]."
> "Only show data by [TIME_RANGE] for everyone except [EXCEPTION]."

Readable by a non-engineer, executable by a machine, quotable in an audit. Its ten masking methods
are a ready-made vocabulary (listed in full in Appendix B), from Hashing — "irreversible sha256 hash, which is consistent for the same value throughout the data source" — to Cell-Level Masking, which "conditionally masks the content in one column based on the value in another column of the same row." Conflict resolution is explicit and teachable: deeper tag hierarchies override shallower ones (PII.SSN supersedes PII); equal depth defaults to earliest-authored; row-level policies merge with AND; reveal exceptions merge with OR.

Camunda's DMN supplies the missing control: input columns, output columns, rules as rows, annotations, and an explicit hit policy — Unique ("Only a single rule can be satisfied or no rule at all"), Any ("all satisfied rules must generate the same output"), First, Rule order, Collect (SUM/MIN/MAX/COUNT). Making "what happens when two rules match?" a visible, named setting is the best idea in the decision-table tradition, so the Business rules screen shows both the policy and which rule won.

Great Expectations contributes mostly — "this rule passes if at least N% of rows satisfy it" — and Dataplex the same idea as a "Passing threshold percentage". Rules that cannot express tolerance get switched off. Monte Carlo contributes governance-by-form-validation: every monitor must carry Priority, Audience, Tags and a Data Quality Dimension (Accuracy, Completeness, Consistency, Timeliness, Validity, Uniqueness), and an account can require any combination at creation. Make the form the policy.

The last piece is the test harness: "show me what Sarah in Region 4 would see" must be one click from the rule editor, or nobody changes a rule after the first week. That is also how the Viewer simulator earns its place as a product surface rather than an admin trick.

For suppression we follow the UK GSS pattern rather than inventing one: primary suppression of small cells, then secondary suppression so the value cannot be recovered by subtraction, then rounding to a base. And suppression must be explicit — <5 with a hover explaining why is trustworthy; a silently missing row is not.

Sources:
<https://documentation.immuta.com/SaaS/govern/secure-your-data/authoring-policies-in-secure/data-policies/reference-guides/data-policies>,
<https://docs.camunda.io/docs/components/modeler/dmn/decision-table-hit-policy/>,
<https://docs.getmontecarlo.com/docs/monitors-overview>,
<https://docs.cloud.google.com/dataplex/docs/auto-data-quality-overview>,
<https://gss.civilservice.gov.uk/wp-content/uploads/2018/03/Guidance-for-tables-produced-from-administrative-sources-4.pdf>

---

7. Discovery for non-technical users

Usage is the documentation. Select Star goes furthest on behavioural metadata. A popularity score is "a metric that indicates how popular your data assets are amongst your team in terms of usage and activity" (service accounts filtered out). A Top Users tab names "people or accounts who run queries on the table the most". A Queries & Joins tab surfaces Popular Queries, Popular Joins ("tables that are most frequently joined to the current table") and Recent Queries. A Related tab separates Related Tables (used together in SELECT queries) from Similar Tables (similar structure). Column-level Field Usage labels

each column "As Is," "Aggr," "Transformed," or "Filter." Databricks' Catalog Explorer leads with Recents, Favorites and Popular. Every dataset in our fixture carries popularity, rank, users and a questions[] array of real phrasings; every field carries a usage label from Select Star's four values.

Requests belong on the object. Atlan puts a Requests tab on the asset profile itself, filterable by status, so asking happens in context and the history stays attached to the thing asked about. Its Activity tab is "a changelog for the asset" — provenance as a tab, not an admin screen — and a README carries free-text human explanation alongside machine metadata. Atlan also bundles access into personas — "A persona scopes what a team sees and can do… It bundles the users and groups on a team, the policies that govern what they can access" — the same
argument as Entra's access packages, in catalogue language.

Purpose is a required field. Collibra models access request as shopping: "Add to Data Basket" from search, review the collected asset cards, then "Check out Data Basket", selecting "the Purpose that describes the business case for which you are requesting access", after which "All data owners must approve the request before you can access the assets." Purview forces the same. Purpose is cheap to collect and it is the field auditors actually read, so it is required on Spiff's request modal — a picked purpose plus a sentence.

Ownership is a person. Owner (accountable), steward (day-to-day) and subject-matter expert are three different roles. Show faces and a contact action, not a mailbox. Every dataset names all three plus a support channel — the fastest route to trust is knowing who to ask.

Sources: <https://docs.selectstar.com/data-discovery/how-can-i-use-this-data>, <https://www.databricks.com/blog/accelerating-discovery-unity-catalog-revamped-catalog-explorer>,
<https://docs.atlan.com/product/capabilities/discovery/concepts/what-are-asset-profiles>, <https://productresources.collibra.com/docs/collibra/latest/Content/Catalog/DataSets/ta_request-access-to-data-set.htm>

---

8. The portal-chat workspace

Cowork is Anthropic's agentic mode for non-developers — it "execute[s] multi-step knowledge work on a user's behalf" for "researchers, analysts, operations teams, legal professionals, finance teams" — framed explicitly as not conversational: "It moves between them, synthesizes information across multiple sources, and completes tasks without the user coordinating each step."

Three panels, three questions. The most useful UI finding in the whole effort.

Panel	Answers	Contents
Progress	what Claude will do	Numbered plan; done steps get a checkmark and strikethrough, current step highlighted
Project	what can Claude see?	Instructions (CLAUDE.md), files, and a Scratchpad audit trail marking each file "wrote to", "viewed", "created"
Context	what can Claude reach?	Uploads (this task only) and Connectors — active integrations only; installed-but-inactive ones do not appear

Separating capability from activity is what makes governance visible rather than claimed. The Context panel's rule matters more than it looks: list only what is live, because an empty slot is itself an honest governance signal. The plan has a documented state machine worth copying rather than reinventing — pending → in_progress → completed (→ deleted) — with a separate activeForm label ("Identifying newsletter threads") shown instead of the static subject while a step runs. Present tense running, past tense done.

The risk dial lives in the composer, not in settings: Manual ("Manually approve"), Auto ("Automatically approve" — works autonomously but "reviews each action for safety"), Skip ("Skip all approvals"), with mid-run steering. Governance is two-layered: that per-task

control, plus org policy at Organization settings > Cowork — "Allow 'Automatically approve' mode", "Allow 'Always allow' for connector tools" (off by default) — with events streamed to SIEM covering "tool calls, file access, human approval decisions". Both layers need mocking; showing one makes governance look either bureaucratic or absent. For a reporting agent the mapping is direct: read-only questions run in Auto, anything that writes or emails drops to Manual.

The share rule we were looking for. Live artifacts are "persistent, interactive HTML dashboards" with version history; sharing stays inside the organisation with "no external or public links", and then:

> "Viewers use their own connector access, not the creator's."

Claude's chat sharing produces a snapshot deliberately full of holes — attachments and "raw data retrieved from MCP tool calls" stay private. Share the narrative; re-run the numbers. And enterprise search ships as a pre-configured project named "Ask Your Org", "starred in your sidebar by default", "Permission-aware: You only see search results from data you have permission to access in the original systems", with "Each user authenticates with their own credentials" — the closest shipped analogue to Spiff's front door.

Sources: <https://www.anthropic.com/product/claude-cowork>, <https://camp-claude.github.io/learn/cowork-task-anatomy/>, <https://code.claude.com/docs/en/agent-sdk/todo-tracking>, <https://support.claude.com/en/articles/14729249-use-live-artifacts-in-claude-cowork>, <https://support.claude.com/en/articles/12489464-use-enterprise-search>

---

9. MCP: the tool list is the permission UI

ThoughtSpot ships a hosted Spotter MCP Server at agent.thoughtspot.app, with OAuth endpoints /mcp and /sse, bearer endpoints /token/mcp and /token/sse, and version pinning via ?api-version=latest | beta | YYYY-MM-DD. Tools: ping, getRelevantQuestions, getAnswer, createLiveboard, getDataSourceSuggestions. Its datasources resource lists "ThoughtSpot Data models the user has access to" — the resource list is itself permission-scoped. The claim is the pitch: "all the permissions and access controls you've already established in ThoughtSpot are automatically respected when your agents access data."

The protocol is clear about who controls what, which is the governance-relevant part.

Primitive	Controlled by	Methods
Tools — functions the model calls to act	Model	tools/list, tools/call
Resources — read-only context, URI-addressed	Application	resources/list, resources/read
Prompts — parameterised templates	User	prompts/list, prompts/get

MCP's own consent expectations are concrete enough to design to: display available tools so users can decide availability per interaction; approval dialogs for individual executions; pre-approval settings for safe operations; and activity logs that show all tool executions with their results. The consent page MUST identify the requesting client by name, display the specific scopes, and show the registered redirect_uri. Scope minimisation is a named risk — wildcard scopes cause "consent abandonment: users decline dialogs listing excessive scopes." A 403 carrying
WWW-Authenticate: Bearer error="insufficient_scope" drives a step-up authorization flow, which means a mid-task permission prompt is normal and deserves a first-class modal.

Claude's connector directory supplies the interaction model: three trust labels — Verified (tested for quality and compatibility, explicitly "not a security audit"), Community, Custom — affecting discovery and display, not capability ("once connected, a community connector has the same capabilities and access as any connector you grant"); per-tool permission in three states, Always allow / Needs approval / Blocked, with a Tool access mode of Auto or On demand; and a request flow rather than a dead end — Request → Requested → an admin Requests tab with a count badge → the outcome shown

back to the requester.

Derived rules for Spiff: split read from write visually everywhere; default every write to Needs approval; render real arguments on the approval card — "Run GST Weekly Summary, filtered to Western Cape, as Innocent Bhengu" is a decision a human can make, "Spiff wants to run a tool" is not; never expose raw query execution; and design for tool drift, since a connector's tool list can change after consent.

The element almost nobody ships, and the one that most suits our story, is the ceiling: an explicit "what this cannot do" block — cannot widen sharing, cannot alter permissions, cannot query outside the semantic model, cannot see a field that is not loaded. Stating the ceiling builds more trust than listing the grants.

Sources: <https://developers.thoughtspot.com/docs/mcp-integration>, <https://modelcontextprotocol.io/docs/getting-started/intro>, /specification/2026-07-28/basic/security_best_practices, <https://claude.com/docs/connectors/directory>, <https://support.claude.com/en/articles/11176164>

---

10. Onboarding: sample data first, your data last

ThoughtSpot's first run is four steps and not a feature tour: "Step 1: Get Started" (overview plus a short video, with Continue and "Exit to homepage" top-right — skippable from screen one); "Step 2: Recommended data source" (introduces one source, pre-selected by the administrator — the new user is not asked to choose); "Step 3: Select a Liveboard"; "Step 4: View your insights", where a Follow button offers to "receive periodic emails about this Liveboard." The flow's job is to end with the user owning one board and subscribed to it, and it is re-runnable via Profile → Experience → "Revisit onboarding experience."

Two sequencing decisions matter more than the steps. Data before asking for data: "Spotter comes pre-populated with sample data, so you can immediately try digging into data analysis" — the "Sales West - Overview" Liveboard exists before you connect anything, and only after a first win does the product offer "Try Spotter on your own data" with three graded on-ramps (CSV, Google Sheets, cloud warehouse). And in the nine-step trial checklist, "invite up to 5 teammates" and "share a Liveboard" fall at the end, as the pay-off, not as setup friction.

The landing surface is a find-first home, not a dashboard: a search bar with a Sources dropdown; a KPI band headed "Track important KPIs" whose empty state reads "Add KPIs to your watchlist" with "Create alert" per card; "Recently viewed"; and a right rail of "Trending Liveboards and Answers" — top 5 with view counts, the cheapest social-proof discovery mechanism on the page. Navigation is permission-shaped: Admin and SpotIQ appear only to users holding the privilege. Hide, do not grey out — the UI is the permission model made visible.

The brand system validates the structure of ours rather than changing it: a near-black navy ground (Dark Spot #08062B) with one electric accent (Cyan Spot #04D1FF), a ladder of blues between (Navy Spot #1B3E61, Blue Spot #346DC9, White Tint #EEF7FF), and six tertiary colours reserved for data (#8AAFFF, #32D9DF, #C493FF, #FF92A8, #FFC052, #6DD267). One ground, one pointer, one chart ramp. The display face is Geist Mono in Medium and Regular only — hierarchy through weight, not size. Spiff's UBT navy/blue with Schibsted Grotesk and IBM Plex is structured identically; we adopted the discipline of one accent doing all the pointing.

Two pieces of positioning we took: name the villain in the sub-head (every persona page names a specific indignity — "waiting for reports or dashboards from your data team"), and sell the gatekeeper their own liberation — self-service "reduces ad hoc requests to data teams, and frees

them to focus on what matters most—governance, models, and high-value analysis." UBT's reporting
team must read Spiff's pitch and see less queue, not less job.

Sources: <https://docs.thoughtspot.com/cloud/latest/getting-started-free-trial>, .../user-onboarding-experience, <https://www.thoughtspot.com/brand>, <https://www.thoughtspot.com/analyst>

---

11. Where Spiff deliberately diverges

1. Sharing never widens access. ThoughtSpot's default column-security mode is Permissive: "when someone shares an object with you, you can see all the data it uses, regardless of explicit permissions to the parent object." Their share dialog patches the gap with a "Give view access to underlying data sources" checkbox, a black warning symbol per recipient who lacks access, and a green checkmark once granted — genuinely the best idea in their flow, but it exists because the default is wrong. Sharing is also transitive: recipients "can further distribute access to others."

Spiff inverts the default. Sharing organises; it never widens; each viewer's copy re-runs scoped to them. The dialog states the consequence rather than offering a checkbox to escape it — "Ana will run this herself. She'll see only her own permitted rows — you may see more" — and shows, per recipient, what they will get: row count, hidden columns, empty-result warning. We keep two things from their model regardless: copy link is not a grant ("Sending users this link does not share the object with them"), and exploring must never mutate the shared thing ("When you apply a filter, the Liveboard is not automatically saved with your filter applied").

2. Read events are logged. Their security stream has full CRUD and one coarse SHARE_OBJECTS,
but no view, no query and no download event; reads live in separate monitoring Liveboards and security events are kept 30 days. A log of successes alone also cannot answer "who is probing what." Spiff's Activity log carries reads and egress beside writes, denials beside grants, and the run-as identity on every row. The vocabulary we designed to includes REPORT_VIEWED, QUERY_EXECUTED,
RESULT_SET_EMPTY_DUE_TO_RLS, DOWNLOAD_COMPLETED (format, rows, columns, bytes),
ACCESS_DENIED_OBJECT / ACCESS_DENIED_COLUMN,
AUTOMATION_CLONED_ON_SHARE,
AUTOMATION_PERMISSION_RECHECK_FAILED,
DELIVERY_SUPPRESSED_EMPTY_FOR_VIEWER,
AI_ANSWER_CORRECTED_BY_USER and IMPERSONATION_STARTED / ENDED. We kept their best instinct —
logging the attempt beside the success — and extended it to denials. Every event carries actor and effective subject as separate fields, because under run-as and preview-as they differ.

3. Preview-as-user is a product surface, not an admin workaround. ThoughtSpot has no in-product "view as user" and no RLS preview; its advice for testing a row-security rule is to "sign in as users in different groups", and impersonation exists only as an API primitive behind CONTROL_TRUSTED_AUTH. Spiff's Viewer simulator is a first-class screen with a loud persistent banner, reachable from the share dialog, the rule editor, the dataset profile and a person's page, because it is the only way to make three promises legible at once: what a rule does, what a share will show, and what "runs as its owner" means. It emits its own audit events, so a preview is never invisible.

Two smaller divergences. Noise control: ThoughtSpot documents no per-alert snooze, only an admin floor ("Shortest time period to check alerts") plus a caveat that a transient breach may never notify; Spiff offers per-alert snooze and cooldown and states the sampling behaviour plainly. Group subscriptions: theirs require that "Users must be removed from the group to be removed from the alert schedule"; Spiff always allows a personal unsubscribe and shows why someone is

subscribed ("via LDM Coordinators").

---

12. Code and documentation as the source of dataset meaning

This pass came later than the rest, after Oren extended the brief: "the system needs to be able to register data sources … then we ask the system to understand datasets by reading the rules from the
code, the logic from the code, understand the database structure from the code and documentation etc." Everything in §3 assumed a semantic layer already existed and somebody had written the definitions into it. This section asks the question underneath: how much of what a dataset means can be read out of the systems that produce it, and where exactly does the machine have to stop and
a person start?

The short answer, across every product and paper we could verify: automation harvests structure and behaviour very well, harvests intent badly, and every serious vendor draws the line in the same place — the machine writes into a suggestion slot, and a human moves it into the fact slot.

What catalogues actually harvest by themselves

Microsoft Purview separates the two halves of the job explicitly. Scanning "connects to the data source and captures technical metadata like names, file size, columns", "extracts schema for structured data sources" and "applies classifications on schemas"; ingestion then "analyzes the input from the scan, applies resource set patterns, populates available lineage information". Three details are instructive. Lineage does not come from the scan at all — it comes from separate lineage connections to Data Factory and Synapse. Classification only samples data at Level-3 scans. And the catalogue has no independent knowledge of reality: "A Microsoft Purview catalog is only aware of the state of a data store when it runs a scan. For the catalog to know if a file, table, or container is deleted, it compares the last scan output against the current scan output." A harvest is a snapshot with a date on it.

Databricks Unity Catalog takes the opposite approach and captures lineage from what actually ran: "Unity Catalog captures lineage automatically for queries run on Azure Databricks, down to the column level, and aggregates it across all workspaces attached to the metastore." It is candid about the ceiling — "Unity Catalog captures lineage to the column level as much as possible. However, there are some cases where column-level lineage cannot be captured" — and publishes the
exclusion list: renamed objects lose their lineage, RDDs and global temp views are not captured, runs submit and spark submit jobs are invisible, and the lineage system tables keep "a rolling 1-year window". Runtime capture knows only what ran; static parsing knows only what was written. Neither is the whole truth, and the difference between them is itself a finding.

Select Star propagates human documentation along lineage rather than generating it, and marks the difference in the interface: "User descriptions and loaded descriptions will be displayed in black, and suggested descriptions will be shown in gray to highlight the origin of the description." Amundsen goes further and makes it structural — a programmatic description is a separate field that "cannot be modified manually" and "would not appear on the page unless it is populated", precisely so machine-written text and human-written text never merge into one unattributable blob. Atlan suggests data-quality rules from asset structure and states the dependency plainly: "The quality of suggestions depends on your asset metadata—column names, data
types, descriptions, database, and schema information." Alation's behavioural metadata — popularity, top users, query patterns — rests on Query Log Ingestion, and OpenMetadata ships an ingestion framework of the same shape, publishing its own fork of the sqllineage parser on PyPI as openmetadata-sqllineage.

The common ground is worth stating flatly. Schema, keys, types, freshness, popularity, joins that

were actually executed: automated, reliable, cheap. What one row means, which rows count, and why a
column is empty for half the estate: none of these are in the schema, and no harvester finds them.

Sources: <https://learn.microsoft.com/en-us/purview/data-map-scan-ingestion>,
<https://learn.microsoft.com/en-us/azure/databricks/data-governance/unity-catalog/data-lineage>,
<https://docs.selectstar.com/features/auto-documentation>,
<https://github.com/amundsen-io/amundsen/issues/147>,
<https://docs.atlan.com/product/capabilities/governance/data-quality/how-tos/use-ai-suggested-rules>,
<https://docs.alation.com/en/latest/datasources/AddDataSources/QueryLogIngestion.html>,
<https://pypi.org/project/openmetadata-sqllineage>

Column-level lineage parsed from SQL

This is the most mature piece of automated semantics in the field, and the one with the best public account of its own failure modes.

DataHub publishes its parser design in full: five steps — parse the SQL to an AST, qualify table names to db.schema.table, fetch the actual table schemas from the metadata store, qualify column references through CTEs, subqueries and wildcards, then emit column lineage edges. "For SQL
parsing we use SQLGlot, with targeted runtime patches applied via the patchy library." The load-bearing argument is step three: "A schema-naive parser can't possibly know where the columns
actually came from." Meaning cannot be parsed out of the text alone; you need the structure beside it. They are equally direct about the limits — "We don't handle things like json_extract, struct fields, or UNNEST-based joins" — and note that "highly dynamic SQL, exotic UDFs, custom macro patterns" need lineage emitted manually through the SDK. They benchmark against a corpus of ~7,000
BigQuery SELECT statements and ~2,000 CREATE TABLE … AS SELECT statements.

SQLLineage is the widely used open-source alternative: "Given a SQL command, SQLLineage will tell you its source and target tables, without worrying about Tokens, Keyword, Identified and all the jargons used by a SQL parser." It parses with sqlfluff and sqlparse and stores lineage in a networkx graph.

OpenLineage's Column Level Lineage Dataset Facet is the most transferable object in this whole pass, because a lineage edge in it carries why, not just what. Each output field lists its inputFields, and each input carries transformations with a type, a subtype, a description and a masking boolean. DIRECT means the "output column value was somehow derived from inputField value", with subtypes IDENTITY ("output value is taken as is from the input"), TRANSFORMATION and AGGREGATION. INDIRECT means the "output column value is impacted by the
value of inputField column, but it's not derived from it" — subtypes JOIN, GROUP_BY, FILTER, SORT, WINDOW, CONDITIONAL. And masking records that values were "obfuscated during the transformation".

That INDIRECT category is exactly where the interesting rules live. A WHERE clause that drops cancelled meetings does not appear anywhere in the output column, and it is the entire reason two teams publish different attendance figures. A lineage model that only records DIRECT edges is blind to the class of thing a reporting agent most needs to explain.

Sources: <https://datahub.com/blog/extracting-column-level-lineage-from-sql/>,
<https://sqllineage.readthedocs.io/>,
<https://openlineage.io/docs/spec/facets/dataset-facets/column_lineage_facet/>,
<https://github.com/OpenLineage/OpenLineage>

Code as the definition of a metric

The strongest version of "read the meaning from the code" is not inference at all — it is putting the definition in the code in the first place and generating the documentation from it.

dbt models the semantics as a graph: a semantic model carries entities, dimensions, measures, a description and a primary entity, and "Think of semantic models as nodes connected by entities in a semantic graph." MetricFlow builds queries from that graph rather than from hand-written SQL. Two artefacts make it machine-readable evidence rather than prose. manifest.json is "a full representation of your dbt project's resources (models, tests, macros, and more), including all node configurations and resource properties", with parent_map and child_map and — for executed nodes
only — compiled_sql. And persist_docs "optionally persist[s] resource descriptions as column and relation comments in the database", pushing the description down into the warehouse so the comment and the model cannot drift apart. Even here the caveats are physical: column-level comments
on Databricks "require file_format: delta (or another 'v2 file format')".

Cube adds a field for the agent that the user never sees: ai_context "provide[s] context to the AI agent without exposing it in the user interface", and must be defined "on views or on individual members (measures, dimensions)" — context defined at the cube level "is not consumed by
the AI agent". It is meant for "which measures to prefer, nuances about data quality, or business logic that would be confusing in a user-facing description."

Looker is the claim to weigh carefully. Google states that "Looker's semantic layer reduces data errors in gen AI natural language queries by as much as two thirds", on the argument that the model should search "clearly defined business objects within LookML (e.g., Orders > Total Revenue)" rather than write SQL against "raw tables with ambiguous field names". The direction is almost certainly right and the number is internal testing with no published methodology; we treat it as a plausible order of magnitude and nothing more.

ThoughtSpot's AI Context (§3) is the same idea with a warning attached that every generate-then-edit feature needs: "If you re-run the 'Generate AI Context' process, the system rewrites all existing AI Context, including your manual edits."

Sources: <https://docs.getdbt.com/docs/build/semantic-models>,
<https://docs.getdbt.com/reference/artifacts/manifest-json>,
<https://docs.getdbt.com/reference/resource-configs/persist_docs>,
<https://docs.cube.dev/docs/data-modeling/ai-context>,
<https://cloud.google.com/blog/products/business-intelligence/how-lookers-semantic-layer-enhances-gen-ai-trustworthiness>,
<https://docs.thoughtspot.com/cloud/26.7.0.cl/spotter-ai-context>

AI-assisted metadata, and the caveats the vendors publish themselves

Databricks ships AI-generated comments in Unity Catalog and documents the limits more honestly than any marketing page would: comments are "powered by a large language model (LLM) that takes
into account object metadata, such as the table schema and column names"; "AI models are not always accurate and comments must be reviewed prior to saving. Databricks strongly recommends human
review of AI-generated comments to check for inaccuracies." There is a hard carve-out — "The model should not be relied on for data classification tasks such as detecting columns with PII" — and a side effect worth knowing: "Saving comments triggers an ALTER SQL command, which can disrupt … pipelines and jobs." The interaction is a two-button decision: "Click Accept to accept the comment as-is, or Edit to modify it before you save it."

Collibra's classification is the most complete acceptance model we found, and three of its behaviours went straight into our thinking. It suggests without asking — "Collibra analyzes a subset of the data in a data source and suggests a data class for that data without human input" — but scores everything, returning classifications only above a configurable floor: "If a data class

specifies a minimum confidence threshold above 0, then the classification is returned only if that threshold is reached." Stewards "accept or reject suggested data classifications manually or automatically." And the system has a memory of refusals: "The automatic data classification process remembers any rejected data class suggestions, meaning a data class will not be suggested
again if you have rejected the data class for an asset." Acceptance is also sticky — "Once a data classification has been accepted for a column, the data classification won't be automatically updated if you run the data classification process again."

Atlan frames its suggestions as a queue with an explicit apply step — review the suggested rules, "adjust thresholds and parameters as needed", select the ones you want, then apply — and adds the line every code-generating feature needs: "Always review and validate Copilot-generated SQL before saving to confirm it matches your requirements."

Four products, four vocabularies, one shape: generate → review → accept or reject → record, with the machine's output visually distinct from the human's until somebody signs it.

Sources: <https://learn.microsoft.com/en-us/azure/databricks/comments/ai-comments>, <https://www.databricks.com/blog/announcing-public-preview-ai-generated-documentation-databricks-unity-catalog>, <https://productresources.collibra.com/docs/collibra/latest/Content/Catalog/DataClassification/UnifiedDataClassification/co_about-data-classification.htm>, <https://docs.atlan.com/product/capabilities/governance/data-quality/how-tos/use-ai-suggested-rules>

Extracting business rules from source code

This is a thirty-year-old research problem, not a new one, and the literature is unusually useful because it measures itself.

The founding paper is Huang, Tsai, Bhattacharya, Chen, Wang and Sun, Business Rule Extraction from
Legacy Code (COMPSAC '96), which combines "variable classifications, program slicing, and hierarchical abstraction" — and rests on an observation that has aged perfectly: organisations trust the code more than the documentation, because the code is what runs.

The modern deterministic tool in that line is COBREX (ICSME 2022, rishalab): "a tool to extract business rules from a COBOL program", which parses with ANTLR4 against the COBOL85 grammar,
performs "COBOL data division analysis, identif[ies] business variables and construct[s] the Control Flow Graph (CFG)", then walks the graph depth-first so that "all statements and their corresponding context statements of a business variable are added to a Rules set." A rule is a statement plus the control flow that guards it — which is precisely why a masking guard, a WHERE clause and a validator are the same kind of object.

The LLM comparison exists and is recent: COBRAIN (Chiranjeevi B S and Sridhar Chimalakonda, IIT
Tirupati, EASE 2025) uses few-shot prompting to extract and summarise COBOL business rules, and
benchmarks itself against COBREX. The numbers deserve to be read carefully rather than cheered:

Measure	Result
COBRAIN vs COBREX	"a precision of 1.0 and a recall of 0.746"
COBRAIN vs manual ground truth	F1 0.73
COBREX vs manual ground truth	F1 0.59
Comprehension study, 28 participants	"over 80% chose COBRAIN over COBREX to have more understandable BRs"

Three readings. The LLM is better than the parser and much more readable — and the best published F1

on this task is 0.73, which means roughly one extracted rule in four is wrong or missing. Perfect precision against COBREX with 0.746 recall says the LLM missed a quarter of what a deterministic parser found, so the two are complementary, not substitutable. And a comprehension win is not a correctness win: people preferred the output that was easier to read, which is exactly the condition under which a wrong rule gets waved through.

Sources:
<https://asu.elsevierpure.com/en/publications/business-rule-extraction-from-legacy-code/>, <https://www.computer.org/csdl/proceedings-article/icsme/2022/795600a464/1JeFivgJU1G>, <https://rishalab.github.io/COBREXdoc/>, <https://dl.acm.org/doi/10.1145/3756681.3756982>, <https://conf.researchr.org/details/ease-2025/ease-2025-research-papers/57/LLM-Vs-Rule-Based-A-Tool-and-Empirical-Study-on-Extracting-Business-Rules-from-COBO>

When the code and the comment disagree

Code-comment inconsistency has its own literature, and its findings bear directly on how much a column comment or a data dictionary is worth as evidence. The state of the art is a purpose-built, fine-tuned model — Rong, Yu, Liu, Tan, Zhang, Shen and Hu, Code Comment Inconsistency Detection and
Rectification Using a Large Language Model (ICSE 2025) — reporting post-hoc detection at "F1 and
Accuracy of 89.0% and 89.6%" against a prior best of 86.4% / 87.3%, and just-in-time detection at "91.4% and 91.8%". Rectification is the sobering half: human evaluation found it produced a correct fix for "65.0% and 55.9% in just-in-time and post hoc, respectively."

So: a dedicated model, on a task it was fine-tuned for, still mislabels roughly one comment in ten and successfully repairs barely more than half of the ones it catches. Anything that reads a free-text comment and treats it as a specification is standing on the weakest available evidence.

Sources: <https://dl.acm.org/doi/10.1109/ICSE55347.2025.00035>, <https://people.cs.umass.edu/~brun/class/2024Fall/CS692P/idllm.pdf>, <https://www.inf.usi.ch/lanza/Downloads/Wen2019a.pdf>

Risks worth naming honestly

1. A rule that compiles is not a rule that runs. Static reading cannot distinguish a live branch from a dead one; only runtime evidence can. Unity Catalog's lineage has the opposite bias — it only knows what actually executed — which is why the two together are stronger than either, and why "no code reads this column" is a finding in its own right rather than an absence.
2. Dead code is a measured failure mode of machine-generated code, not a hypothetical. Liu et al.'s taxonomy of hallucinations in LLM-powered code generation labels 3.2% of their sample as Dead Code, alongside Intent Conflicting at 32.1% and Context Inconsistency at 31.8%, having located 2,119 hallucinations across 3,084 samples. That study is about writing code, not reading it, and we could not verify a published rate for code comprehension — the corresponding code-summarisation hallucination paper was behind a paywall. Treat the direction as established and the magnitude as unknown.
3. Test fixtures are not populations. A test is strong evidence of intended behaviour and the invented data inside it is evidence of nothing. Reading a fixture as a distribution, or a fixture's names as members, is a category error. We found no published treatment of this; [SYNTHESIS].
4. Documentation that contradicts the code is common, hard to detect and harder to fix automatically — see the numbers above. The right product response is to surface the contradiction, not to resolve it.
5. Classification from a sample is a probability, not a fact. Purview samples only at Level-3; Collibra classifies from "a subset of the data". Both are honest about it. A UI that renders a sampled classification as a settled label is not.
6. Do not let a machine decide what is personal data. Databricks says it outright.
7. Re-running the generator can destroy human work. ThoughtSpot's regeneration warning is the

canonical example. Any second pass must diff against what a person accepted, not overwrite it.
8. An unreviewed derived rule is more dangerous than no rule at all. A missing rule produces a visible gap that somebody notices. A wrong rule produces a confident number that nobody questions, propagates into every answer built on the dataset, and is defended by the fact that the system said so. This is the whole argument for a review queue, and it is why the acceptance step must be a person with a name rather than a confidence threshold.

Design lessons for Spiff

1. Read structure and logic; never read the rows. The verifiable prior art harvests schema, keys, lineage and query behaviour. Everything Spiff needs beyond that lives in migrations, models, validators, tests and documents. Where a read has to touch data at all — a null count, a row count — label it on the finding, on the evidence and in the log.
2. A finding is a claim plus its evidence, and the evidence is a location. Every product that does this well anchors the machine's output to something a human can open. File, migration, page, schema object, probe. No anchor, no finding.
3. The machine writes into a suggestion slot; a person moves it into the fact slot. Amundsen separates the fields, Select Star separates the colours, Databricks separates the buttons. Nothing proposed should be indistinguishable from something agreed.
4. Score every finding, and let the score gate presentation rather than admission. Collibra's confidence floor decides what is shown; a person still decides what is true. Weak findings must look weak — a number alone will not do it.
5. Remember refusals. Collibra never re-suggests a rejected class for the same asset. A rejection with a reason is training data for the next run and an answer for the next reader; a rejection without one is a decision that has to be taken again from scratch.
6. Make acceptance sticky and re-runs non-destructive. An accepted definition must survive the next scan. The second run's job is to show the diff against what a person agreed, not to overwrite it.
7. Model the indirect edge, not just the derived one. OpenLineage's INDIRECT — filters, joins, group-bys, conditionals — is where the rules that change a published number actually live, and where a masking flag belongs.
8. Prefer the definition that was written down over the definition that was inferred. dbt's persist_docs and semantic models are the strongest form of this. Where UBT has a dbt project, read the model; where it does not, read the code and say so.
9. Expect roughly one rule in four to be wrong. F1 0.73 is the best published figure on rule extraction. Design the queue for that rate: bulk acceptance for structural findings, individual scrutiny for anything that changes a number, and a named approver on every one.
10. Deterministic parsing and language models are complementary. Precision 1.0 with recall 0.746 says the model is confident and incomplete. A real build should parse the schema and the SQL deterministically, use the model for meaning and prose, and treat agreement between the two as the actual confidence signal.
11. Two authoritative sources that disagree are a decision, not a calculation. Show both, name the difference in units a reader cares about, and route it to the person who owns the number. Neither picking a winner nor hiding the conflict is defensible.

What we could not verify in this pass

Alation's Query Log Ingestion page did not render for us; the popularity, top-user and query-pattern claims attributed to QLI are carried over from §7 and from connector documentation, not from the QLI
page itself. OpenMetadata's lineage documentation returned 404 at the version we tried, so its column-level lineage behaviour is described only at the level the ingestion framework and the openmetadata-sqllineage package support. DataHub publishes benchmark charts against sqllineage
and openlineage-sql but the figures are in an image we could not transcribe, so we quote the

corpus size and not the win margin. Marquez's column-lineage endpoint is under active development in
the issue tracker and we did not confirm its current shape. Google's "two thirds fewer errors" is internal testing with no published method. Wen et al.'s large-scale code-comment inconsistency study
is cited for its existence; we could not open it to quote its numbers, so every figure in that subsection comes from the ICSE 2025 paper instead. And the ACM paper on hallucinations in LLM-based code summarisation (10.1145/3808139) was paywalled, which is why the hallucination figures above are from a code generation study and are flagged as such.

---

13. What we built as a result

Each row is a finding that produced a specific screen or component. v1 screens marked.

#	Finding (source)	v2 screen or component
1	Interpretation as the answer title; token grammar colour-coded — measures green, attributes blue, filters grey (ThoughtSpot)	Answer (v1): parsed-question chip row above the chart, using .tok.m / .a / .f
2	Highlight what changed in the tokens on each follow-up (Spotter)	Answer (v1) token diff; Portal chat emphasises changed chips per turn
3	Analysis plan before execution (Spotter "Why"); reviewable todo list (Cowork)	Portal chat: plan panel using pending / in_progress / completed with an activeForm verb phrase
4	Three panels answer will-do / can-see / can-reach (Cowork)	Portal chat: Plan · Definitions & instructions · Reachable data (active connectors only)
5	Approval mode in the composer; org policy gates the modes (Cowork)	Portal chat composer control; org defaults on Connectors
6	Approval cards must render real arguments (synthesis)	Portal chat: card naming report, filter and run-as identity
7	"Viewers use their own connector access, not the creator's" (Cowork live artifacts)	Share dialog copy on Answer, Library, Team spaces, Dashboards (v1)
8	Object and data access are separate axes, shown per recipient; copy link is not a grant (ThoughtSpot)	Share dialog: per-recipient scope preview — rows visible, columns masked, empty-result warning — plus a link disclaimer
9	Personalised views keep filter state without forking; unsaved-change dot then Update (ThoughtSpot)	My workspace (v1): saved views of a shared answer, with the dot-and-Update state model
10	Four trust states; warning is not deprecated (Alation, Atlan)	CERT vocabulary; badges on Data catalogue, Dataset profile, Home
11	Verification decays — the post-edit re-verify banner (ThoughtSpot)	Dataset profile: certification block with verifier, date, stale-since-edit banner
12	Permanent provenance: last refresh, creator, sample values (ThoughtSpot tooltip)	Dataset profile: freshness strip — refreshed, next, sla, freshness, row-count sparkline
13	Recommendations beat scores — but give a number too (Spotter optimization)	Dataset profile: quality score plus completeness / validity / freshness / uniqueness
14	Dependents tab listing objects and the columns they use (ThoughtSpot)	Dataset profile: "What uses this" — joins, saved answers, automations
15	Synonyms are the cheapest discovery feature (ThoughtSpot columns)	Field synonyms[] on Dataset profile, surfaced in Ask (v1) suggestions
16	Attribute vs measure; field usage labels As Is / Aggr / Transformed / Filter (ThoughtSpot, Select Star)	Dataset profile: type chips (attribute / measure / date / geo) and a per-field usage column
17	Popularity, top users, popular questions, popular joins (Select Star, Databricks)	Data catalogue ranking; Dataset profile "How people use it"

18	Sample data rendered under the viewer's own policies (synthesis)	Dataset profile: masked preview rows with a banner naming what is hidden and why
19	Governed definitions as prose with an owner and version (ThoughtSpot, dbt, Cube)	Definitions / glossary: term, def, owner, version, agreed, used
20	Rules as sentences with dropdowns (Immuta)	Business rules: the .sentence + .slot builder
21	Named hit policy for overlaps (Camunda DMN)	Business rules: HIT_POLICIES, precedence rank, "which rule won"
22	Tolerance is normal — mostly, passing threshold (Great Expectations, Dataplex)	Business rules: tolerance field and pass-rate readout
23	Governance metadata enforced at creation (Monte Carlo)	Business rules: owner, approver, category, severity required before save
24	A rule needs a test harness (synthesis)	Viewer simulator, launched from the rule editor
25	Primary then secondary suppression, then rounding to a base (UK GSS)	Small-count rules; <5 cells with an explanatory hover
26	Roles bind to groups, never individuals (ThoughtSpot RBAC/RLS/CLS)	People & access: group and bundle model; person rows read-only for entitlements
27	Bundle access, time-bound, attribute-driven (Entra)	ROLES[] bundles with plain-English descriptions and risk level; GROUPS[] with type and rule
28	Effective-permissions inspector naming the granting group (synthesis)	People & access → person: resolved privileges with source group
29	Review the diff; dormancy is the highest-value column (SailPoint)	People & access: review queue, change-since-last-review, dormant and suspended flags
30	Split download into separate privileges; scope the directory itself (ThoughtSpot)	Bundles distinguish "Export summary" from "Export detail"; PEOPLE_HIDDEN states the 396 unlisted people
31	Purpose is a required field (Collibra, Purview)	requestAccessModal(datasetId): purpose picker plus justification
32	Approved and Completed are different statuses (Purview)	My data access: Pending / Declined / Approved / Completed
33	Requests belong on the object; a requester screen and an approver queue (Atlan, Purview, Entra)	Dataset profile request history; My data access; approver queue on People & access
34	Log reads, egress, denials and attempts (ThoughtSpot gap)	Activity log: AUDIT_KINDS spanning read, egress, denial, automation, AI
35	Actor and effective subject as separate fields (synthesis)	Activity log: run-as column on every row
36	No in-product "view as user" (ThoughtSpot gap)	Viewer simulator with persistent banner and its own audit events
37	The tool list is the permission UI, shown before connecting (MCP, Claude)	Connectors: full tool table, plain English, read/write split
38	Three permission states; writes default to Needs approval (Claude)	Connectors: the .tri three-state control
39	State the ceiling — what this cannot do (synthesis)	Connectors: "What Spiff will never do through this connector"
40	Honest trust labels ("not a security audit"); design for tool drift (Claude, MCP)	Connectors: badge with inline explainer; "this connector added N tools — review" banner
41	Destinations / Pipelines / Activity as three tabs (ThoughtSpot Sync)	Automations and Triggers & schedules (v1) reorganised on that split
42	Gating conditions; per-recipient scope at schedule time; an individual exit from a group subscription (ThoughtSpot)	Triggers & schedules (v1) plain-language gate; Automations (v1) recipient scope list, personal unsubscribe, "subscribed via LDM Coordinators"
43	Onboard to one owned thing; ship data before asking for data; skippable and re-runnable	Onboarding & registration: four steps ending in one report the user owns and follows, on sample data
44	Home is a find surface — search, watchlist, recent, trending; permission-shaped nav	Home: what colleagues run, each item re-run per viewer; left rail rendered from the viewer's bundles
45	Chart stability: pick once, keep on follow-ups, keep colours in a session	Answer (v1) reshape bar: eligibility from data shape, stability rule added

| 46 | Capability catalogue as a permission-shaped list (ThoughtSpot custom actions, Cowork plugins) | Capabilities (v1): unavailable capabilities shown with the reason |
| 47 | "No data" and "not allowed to see the data" must look different (MCP) | Empty states across Answer, Dataset profile, Data catalogue; the blocked state |

---

14. Confidence notes

Each source pass flagged what it could not verify. Those flags travel with the findings.

ThoughtSpot — search and AI. Per-chart-type data-shape rules are not published (types only). How the "Change visualization" picker renders unsupported types is unknown. Conditional-formatting
microcopy is unverified — the page 404'd. The keyword reference was not enumerated in full. Exact composer placeholder and empty-state copy are unverified. Whether "Deep analysis mode" and "Research mode" are one surface under two names is unclear: the URL says spotter-research-mode,
the body says "Deep analysis mode".

ThoughtSpot — Liveboards and sharing. A developer-doc line ("When an object is shared, users can view all the data regardless of the permissions set at the parent object level") appears to concern parent-object permissions rather than RLS; the exact interaction is unverified. Microsoft Teams as a native alert channel is unverified, as is the absence of a per-alert snooze and of any Liveboard-level "Follow" distinct from schedules and the watchlist.

ThoughtSpot — modelling. No per-connection health indicator was found. No "verified Model" badge exists — verification is documented for Liveboards only. No numeric model health or quality score exists, so Spiff's dataset quality score is our invention, not an observed pattern. Data workspace tab labels vary between doc versions. The formula function reference was not enumerated; lesson_plans and column_groups have no UI documentation.

ThoughtSpot — governance. Column masking (as opposed to hiding) in CLS is unverified, as are the exact Users-list columns and filtering, whether impersonated sessions are distinguishable in audit logs, and Audit Logs API rate limits. Explicitly not ThoughtSpot features: the extended audit vocabulary in §11, the bulk-admin patterns and the design conclusions — those are our reasoning.

ThoughtSpot — site and brand. In-product screens are reconstructed from documentation, not a live trial. The brand page names no body or UI typeface, and gradient and motion specifics are undocumented.

Cowork and portal chat. The three-panel anatomy comes from one detailed third-party source (Camp Claude), corroborated in outline by DataCamp and aimaker — treat the exact panel headings
as strong but not first-party. That matters, because it is finding #4 in the build table. Message-action labels, keyboard shortcuts, the usage-indicator UI and Research's in-progress rendering are undocumented in vendor help content. The product has oscillated between "Chat / Cowork / Code tabs" and mode-in-composer; we designed to the composer version.

Catalogues, rules and access. Everything marked [SYNTHESIS] in that pass — Appendix A, Appendix B, the eighteen worked example rules, and the patterns for hundreds of users — is our inference. The vendor facts are the quoted labels, statuses, masking methods and hit policies.

The fixture data. Every number in 50-org.js is invented: 428 users, 18.4 M attendance rows, a 97 quality score. Internally consistent and plausible; not measurements.

---

15. Open questions

1. Does the parsed-question chip row actually get read? Our explainability bet rests on a

locality secretary noticing that the timeframe chip says "last quarter" when they meant "this quarter". Test: hand people a deliberately mis-parsed answer, with and without the chip row, and time how long it takes to spot.
2. Is the plan panel reassuring or intimidating? Both precedents serve more technical audiences. A six-step query plan may read as "this is complicated and I might break it."
3. Do four trust states survive contact with real stewards, or does everything become verified within a month because draft looks like an accusation?
4. What is the right unit of certification — the dataset or the answer? If a certified dataset produces an uncertified answer that reaches a divisional manager, whose badge was that?
5. Will anyone author a rule in the sentence builder? Immuta's users are privacy engineers; ours are stewards with a day job. The realistic split may be stewards read and test, a platform admin writes — which is a different screen.
6. Does request-with-purpose get used, or routed around? The failure mode is a phone call and a spreadsheet. Measure requests submitted against grants made outside Spiff.
7. How much masking can a person tolerate before they stop trusting the tool? Three masking rules already apply to our current user on Member records. There is a point where a report full of withheld cells reads as broken rather than governed.
8. Is the Viewer simulator safe to give Regional Coordinators, or admin-only? It is our most persuasive governance demo and also a way to learn what colleagues can see. Logging every simulation is a guess at the right trade.
9. What cadence do people actually want, and does previewing the literal next send remove the anchoring surprise ("your time zone's start of day (00:00)", not creation time)?
10. Does "sharing clones the automation" confuse people? It is the least intuitive invariant, and personalised-view patterns elsewhere set the opposite expectation. A lineage chip ("copied from Ronit's GST weekly, 3 Mar") may not be enough.
11. Is a quality score honest? We invented it; ThoughtSpot deliberately did not ship one. Test whether a 97 changes behaviour more than a three-item punch list.
12. Where does the reporting team see itself in this? The gatekeeper-liberation argument is the strongest thing on ThoughtSpot's site and completely untested at UBT.

Appendix A — the canonical dataset profile page

An ordered inventory composited from Atlan, Collibra, Alation, Purview, Databricks, Select Star and Dataplex. The order is the argument: a non-technical reader goes top-down and should be able to stop at any point with a defensible decision. Marked [SYNTHESIS] in the source pass — the elements are observed, the ordering is ours. It drove the DATASETS[] field list in src/data/50-org.js and the tab structure of the Dataset profile screen.

A. Identity band — 1 business name in plain English, not dim_mtg_attnd_f; 2 technical name or alias, smaller; 3 one-sentence purpose (what question this answers); 4 trust badge (Verified / Draft / Warning / Deprecated, with colour, icon and text); 5 sensitivity badge (Public / Internal / Restricted / Personal); 6 domain chip; 7 actions — Ask a question, Star, Copy link, Share, Request access, Report a problem.

B. Trust and freshness strip — 8 last refreshed (relative and absolute) and next expected refresh; 9 freshness against SLA (on time / late / stale); 10 quality score with dimension breakdown — completeness, validity, freshness, uniqueness; 11 open incidents, linked; 12 row count
and trend sparkline.

C. Stewardship — 13 data owner (accountable): name, photo, team, contact action; 14 data steward (day-to-day), a separate person; 15 subject-matter expert(s); 16 escalation path or support channel; 17 certification date and who certified it.

D. What is in it — 18 field list: business label, technical name, type, description, example value; 19 per-field sensitivity tag and masking state ("you see this hashed"); 20 per-field usage label (As Is / Aggregated / Transformed / Filter); 21 per-field completeness, distinct count, null

rate; 22 linked glossary term per field; 23 search within fields.

E. Sample data — first N rows rendered under the viewer's own policies, with a banner naming what is masked and why. This is where "runs as its owner" becomes visible rather than asserted.

F. Scope and coverage — 24 grain statement ("one row per member per meeting occurrence"); 25 time coverage and geographic or regional coverage; 26 known exclusions and caveats, in prose; 27 population definition — who is in scope and who is not.

G. Rules in force — the governance rules applying to this dataset for this viewer: masking, row filters, suppression thresholds, retention. Each links to the rule object.

H. How people use it — 28 popularity score and rank within domain; 29 top users, with a "people like you" filter on region and role; 30 popular questions; 31 popular joins and commonly combined datasets; 32 saved answers and automations built on it.

I. Lineage — upstream sources and downstream consumers, business-level by default with a technical toggle; column-level on demand.

J. Documentation — README or rich text, linked policy documents, FAQ, glossary terms.

K. Access — 33 your current access level, stated plainly; 34 who else has access (count and roles, not necessarily names); 35 request-access CTA with purpose selection; 36 your request history for this asset.

L. Activity / changelog — schema changes, description edits, certification changes, rule changes, each with actor and timestamp.

Appendix B — the canonical business rule object

Every field a configurable Spiff rule needs, composited from Immuta, Camunda DMN, Monte Carlo, Dataplex, Great Expectations and GSS disclosure guidance. Marked [SYNTHESIS] in the source pass.
It drove src/data/51-rules.js and the Business rules screen.

Field	Purpose
Rule ID	Stable reference, quotable in an audit
Name	Human and imperative — "Mask member email outside own locality"
Plain-English statement	The Immuta sentence. The primary representation, not a summary of one
Category	Masking / Row filter / Suppression / Retention / Metric definition / Consent
Governance dimension	Privacy, Safeguarding, Accuracy, Retention
Scope	Datasets, fields, domains or tags the rule binds to
Condition	Structured predicate, rendered as if/then
Exception clause	"for everyone except…" — groups, attributes, roles
Action	Mask (with method), filter, suppress, round, block, warn, annotate
Severity	Block / Redact silently / Redact with notice / Warn / Log only
Precedence	Explicit rank plus a named hit policy for overlaps
Owner (accountable)	A named person, not a team inbox
Approver(s)	Who signed it off, and when
Status	Draft / In review / Active / Suspended / Retired
Effective from / to	Time-bound rules expire rather than rot
Legal or policy basis	Link to the actual policy document
Last run / evaluations today	Proves it is alive
Pass rate / rows affected	Proves it is doing something
Test and preview	"Show me what Sarah in Region 4 would see" — one click from the editor
Change history	Who edited, what changed, and why
Linked incidents	Where it fired and something broke

Masking methods in the method dropdown, from Immuta: Hashing, NULL, Constant, Regex, Rounding,
Format Preserving Masking, Randomized Response, Reversible Masking, Custom Function, Cell-Level
Masking.

Hit policies for overlapping rules, from Camunda DMN: Unique, Any, First, Rule order, Collect (SUM / MIN / MAX / COUNT).

Worked examples for the GST domain. The source pass produced eighteen; these four carry the mockup, because each demonstrates a different mechanism.

1. Region scoping — default deny. Only show rows where region is in the viewer's assigned_regions, for everyone except National Office. Highest precedence; merges with AND against every other row filter.
2. Mask member email and mobile outside own locality. Mask columns tagged Contact.Direct using Hashing for everyone except users whose locality matches the row's, or who hold Regional Coordinator. Severity: redact with notice.
3. Pastoral notes are never queryable. Any field tagged Pastoral.Confidential is excluded from Spiff entirely — not masked, not listed. Severity: block; the field does not appear in the field list. This is why the blocked certification state exists.
4. Attendance definition is single-sourced. "Attendance" means a member with a confirmed check-in at a meeting of type Regular or Special; excludes cancelled meetings and duplicate check-ins within four hours. Owner: Head of Statistics; version 3, agreed 12 April 2026. Category: metric definition — a rule that changes a number, not a permission, which is exactly why it belongs in the same screen as the masking rules.